

Plan de Laboratorio

Practica 3 - Sistemas Informaticos II

Guia paso a paso para la sesion de lab

Ejercicio	Descripcion	Puntos	Prioridad
Ej1	Plan pruebas P1-base + Aggregate Report	Aprobado	1 (critico)
Ej5	Curva productividad 3 proyectos	1.0	2 (alto)
Ej2	Generalizar plan a 3 proyectos (JMX)	0.5	Ya hecho
Ej6	gunicorn vs runserver	0.5	3 (medio)
Ej7	2 workers + 2 CPUs + SESSION_ENGINE=db	1.0	4 (medio)
Ej3	Test estres 1000 threads + nmon	0.5	5 (bajo)
Ej4	Cuando comienza el 2o loop	0.5	Ya hecho
C1	Diferencias Thread Group P1-base vs P1-ws	0.5	Ya hecho
C2	Donde cambiar SESSION_ENGINE en ws	0.5	Ya hecho

Lo que ya esta hecho: P3_P1-base.jmx, P3-projects.jmx, scripts de automatizacion, respuestas teoricas (Ej4, C1, C2), generate_memoria.py

Lo que falta (requiere lab): ejecutar pruebas, recoger datos, capturas de pantalla, rellenar secciones [PENDIENTE] de la memoria

FASE 0: Preparacion antes del lab (en casa)

Tiempo estimado: 30 min en casa

0.1 Preparar un USB con todo el material

- Copia toda la carpeta **Practica 3 SI 2/** a un USB
- Asegurate de que incluye: **apache-jmeter-5.6.3/**, los .jmx, scripts/, si2_alumnos-main/
- El USB debe tener al menos 500MB libres

0.2 Verificar que tienes todo el codigo

- si2_alumnos-main/P1-base/ (completo, con views.py, pagoDB.py, templates, data.csv)
- si2_alumnos-main/P1-ws-frontend/ (con pagoDB.py que usa requests)
- si2_alumnos-main/P1-ws-backend/ (con views.py usando DRF)
- si2_alumnos-main/P1-rpc-frontend/ (con pagoDB.py que usa xmlrpc)
- si2_alumnos-main/P1-rpc-backend/ (con server_mq.py, models, etc.)
- P3_P1-base.jmx
- P3-projects.jmx
- scripts/run_all_lab.sh, run_productivity_curve.sh, run_exercise3.sh, enable_threadgroup.sh

0.3 Estructura de las VMs (recordatorio)

VM	Rol	Donde corre	Proyectos que la usan
vm1	Base de datos PostgreSQL	PC1 (o PC2)	Todos
vm2	Servidor Django (gunicorn)	PC1 (P1-base) PC2 (ws/rpc)	Todos
vm3	Frontend (ws/rpc client)	PC1	Solo P1-ws y P2-rpc

IMPORTANTE: Para P1-base solo necesitas 1 PC (vm1 + vm2). Para P1-ws y P2-rpc necesitas 2 PCs: en uno corren vm2+vm1 (servidor+BD), en otro vm3 (frontend). NO ejecutes las 3 VMs en el mismo PC, los resultados seran incorrectos.

FASE 1: Setup en el laboratorio

Tiempo estimado: 20-30 min

1.1 Copiar material al PC del lab

- Conecta el USB al PC del lab
- Copia la carpeta **Practica 3 SI 2/** al escritorio o a una carpeta conocida
- Verifica que JMeter esta en la carpeta: **apache-jmeter-5.6.3/bin/jmeter.sh**

1.2 Verificar Java

JMeter necesita Java 8+. Abre un terminal en el PC del lab y ejecuta:

```
java -version
```

Si no hay Java, pregunta al profesor. Los PCs del lab suelen tenerlo instalado.

1.3 Arrancar las VMs para P1-base

- Abre **VirtualBox** en el PC del lab
- Importa o abre las VMs que os dio el profesor (si no estan ya importadas)
- Arranca **vm1** (base de datos) - espera a que arranque completamente
- Arranca **vm2** (servidor) - espera a que arranque completamente

TIP: Puedes verificar que vm1 y vm2 se ven entre si haciendo ping desde vm2 a vm1.

1.4 Copiar el codigo a las VMs

Necesitas copiar el codigo Django a vm2 (y a vm3 cuando toque). Opciones:

- **Opcion A - SCP desde el PC del lab:** Necesitas saber la IP de vm2
- **Opcion B - Carpeta compartida de VirtualBox:** Si las VMs tienen Guest Additions
- **Opcion C - Git:** Si el repo esta en GitHub/GitLab, haz git clone desde dentro de la VM

Para SCP (la opcion mas comun):

```
# Desde el PC del lab, averigua la IP de vm2:  
# En vm2 ejecuta: ip addr show o hostname -I  
  
# Luego desde el PC del lab:  
scp -r si2_alumnos-main/P1-base/ usuario@IP_VM2:~/
```

IMPORTANTE: Pregunta al profesor cual es el usuario y password de las VMs. Normalmente es ubuntu/ubuntu o alumno/alumno.

1.5 Configurar el acceso desde JMeter a vm2

JMeter corre en el PC del lab y necesita acceder al servidor Django en vm2. Hay dos formas:

Opcion A: Acceso directo por IP (RECOMENDADO)

- Averigua la IP de vm2 (en vm2: hostname -I)
- Django corre en el puerto 8000 de vm2
- En los JMX hay que cambiar **host=localhost** y **port=18000** por **host=IP_VM2** y **port=8000**

Para cambiar host y port en los JMX sin editar a mano, usa este comando:

```
# Ejemplo: si la IP de vm2 es 192.168.56.102  
# Editar las variables locales del Thread Group P1-base en P3_P1-base.jmx:
```

```
# host = 192.168.56.102
# port = 8000

# Puedes hacerlo con JMeter GUI o con sed:
# (Ajusta la IP real cuando la sepas)
```

Opcion B: Port forwarding en VirtualBox

- En VirtualBox > vm2 > Settings > Network > Advanced > Port Forwarding
 - Crear regla: Host Port **18000** -> Guest Port **8000**
 - Asi JMeter accede a **localhost:18000** y VirtualBox lo redirige a vm2:8000
 - Si usas esta opcion, los JMX ya estan configurados correctamente (host=localhost, port=18000)
- IMPORTANTE:** Si usas acceso directo por IP, DEBES actualizar host y port en ambos JMX. Si usas port forwarding, los JMX ya estan bien con localhost:18000/18003/18005.

FASE 2: Desplegar y arrancar P1-base

Tiempo estimado: 15-20 min

2.1 Verificar la BD en vm1

Conectate a vm1 por SSH y verifica que PostgreSQL esta corriendo:

```
ssh usuario@IP_VM1
sudo systemctl status postgresql
```

Si no esta corriendo:

```
sudo systemctl start postgresql
```

2.2 Copiar P1-base a vm2 (si no lo hiciste antes)

```
# Desde el PC del lab:
scp -r si2_alumnos-main/P1-base/ usuario@IP_VM2:~/P1-base/
```

2.3 Configurar y poblar la BD

```
# En vm2:
ssh usuario@IP_VM2
cd ~/P1-base/

# Instalar dependencias si es necesario:
pip install -r requirements.txt

# Migraciones y poblar BD:
python manage.py migrate
python manage.py populate
```

TIP: El comando populate carga las 1000 tarjetas del data.csv en la BD. Sin esto, todas las peticiones de JMeter fallaran porque no encontrara las tarjetas.

2.4 Arrancar gunicorn en vm2

```
# En vm2, dentro de ~/P1-base/:
gunicorn visaSite.wsgi:application --bind 0.0.0.0:8000
```

El **--bind 0.0.0.0:8000** es crucial: permite conexiones desde fuera de vm2 (tanto del PC del lab como de vm3).

2.5 Verificar que funciona

Desde el PC del lab, abre un navegador y accede a:

```
http://IP_VM2:8000/visaApp/tarjeta/
# 0 si usas port forwarding:
http://localhost:18000/visaApp/tarjeta/
```

Deberias ver el formulario de tarjeta. Si lo ves, el servidor esta funcionando correctamente.

FASE 3: Ejercicio 1 - Prueba basica P1-base (APROBADO)

Tiempo estimado: 10-15 min

IMPORTANTE: Este ejercicio es OBLIGATORIO para aprobar. Hazlo primero.

3.1 Ajustar la ruta del CSV en el JMX

El CSV Data Set Config en P3_P1-base.jmx apunta a una ruta relativa. JMeter busca el CSV relativo al directorio desde donde lo ejecutas. Asegurate de ejecutar JMeter desde la carpeta **Practica 3 SI 2/**.

```
# Ruta actual en el JMX:
# si2_alumnos-main/P1-base/visaApp/management/commands/data.csv

# IMPORTANTE: ejecuta JMeter desde la carpeta 'Practica 3 SI 2/'
cd 'Practica 3 SI 2/'
```

TIP: Si la ruta del CSV no funciona, puedes cambiarla a una ruta absoluta editando el JMX o abriendo JMeter GUI.

3.2 Ejecutar la prueba

```
mkdir -p results_ej1
rm -rf results_ej1/output
./apache-jmeter-5.6.3/bin/jmeter.sh -n -t P3_P1-base.jmx \
-Jusers=10 -Jsamples=10 \
-l results_ej1/results.jtl \
-Jsummariser.name=summary \
-e -o results_ej1/output
```

3.3 Verificar resultados

- Abre **results_ej1/output/index.html** en el navegador
- Ve a la seccion **Aggregate Report** (o Statistics)
- Verifica que **Error % = 0.00%** (o muy bajo)
- Si hay errores altos, revisa View Results Tree para diagnosticar

3.4 CAPTURA DE PANTALLA

IMPORTANTE: HAZ UNA CAPTURA DE PANTALLA DEL AGGREGATE REPORT. Esta captura va en la memoria y es requisito del Ejercicio 1.

- Guarda la captura como **captura_ej1_aggregate.png**
- Anota los valores de Throughput Total para la memoria

FASE 4: Ejercicio 5 - Curva productividad P1-base

Tiempo estimado: 20-30 min

IMPORTANTE: Este es el segundo requisito para aprobar: obtener al menos UNA curva de productividad.

4.1 Ejecutar el script de curva de productividad

```
chmod +x scripts/*.sh
./scripts/run_productivity_curve.sh P3_P1-base.jmx p1base_gunicorn
```

El script ejecuta JMeter con 1, 2, 3, 5, 7, 10, 15, 20, 25, 30 usuarios, 10 samples cada uno, y extrae el throughput de cada ejecucion.

4.2 Verificar resultados

- El script genera **results_p1base_gunicorn/throughput_summary.csv**
- Mira el CSV: deberia tener una fila por cada numero de usuarios
- El throughput deberia crecer al principio y luego estabilizarse o bajar

```
cat results_p1base_gunicorn/throughput_summary.csv
```

4.3 Apuntar los datos

Copia el contenido del CSV o haz captura. Estos datos van en la memoria como tabla + grafica.

TIP: Con esto ya tienes los 2 requisitos minimos para aprobar (Ej1 + 1 curva). Ahora puedes continuar con el resto con mas tranquilidad.

FASE 5: Ejercicio 5 - Curvas P1-ws y P2-rpc (necesitas 2 PCs)

Tiempo estimado: 40-60 min

5.1 Configurar el segundo PC

Para P1-ws y P2-rpc necesitas 2 PCs fisicos. La distribucion es:

	PC 1	PC 2
VMs	vm3 (frontend)	vm2 (servidor) + vm1 (BD)
Corre	ws-frontend / rpc-frontend	ws-backend / rpc-backend
JMeter	Se ejecuta aqui	-

5.2 Desplegar P1-ws-backend en vm2 (PC2)

```
# Copiar el codigo al PC2 y luego a vm2:
scp -r si2_alumnos-main/P1-ws-backend/ usuario@IP_VM2:~/P1-ws-backend/

# En vm2:
cd ~/P1-ws-backend/
pip install -r requirements.txt
python manage.py migrate
python manage.py populate
python visaSite.wsgi:application --bind 0.0.0.0:8000
```

5.3 Desplegar P1-ws-frontend en vm3 (PC1)

```
# Copiar codigo a vm3:
scp -r si2_alumnos-main/P1-ws-frontend/ usuario@IP_VM3:~/P1-ws-frontend/

# En vm3, editar el fichero env o settings.py:
# RESTAPIBASEURL debe apuntar a vm2:
# RESTAPIBASEURL=http://IP_VM2:8000/restapiserver/

cd ~/P1-ws-frontend/
python visaSite.wsgi:application --bind 0.0.0.0:8000
```

IMPORTANTE: El frontend WS necesita poder conectarse al backend. Edita RESTAPIBASEURL en settings.py o en el fichero env de P1-ws-frontend para que apunte a la IP real de vm2.

5.4 Verificar que funciona

```
# Desde PC1, en el navegador:
http://IP_VM3:8000/visaAppWSFrontend/tarjeta/
```

5.5 Ajustar JMX y ejecutar curva P1-ws

En P3-projects.jmx, el Thread Group P1-ws tiene host=localhost y port=18003. Ajusta estos valores para que apunten a vm3 (donde corre el frontend).

```
# Habilitar solo P1-ws:
./scripts/enable_threadgroup.sh P1-ws

# Ejecutar curva:
./scripts/run_productivity_curve.sh P3-projects.jmx p1ws_gunicorn
```


5.6 Repetir para P2-rdp

- Para en vm2 el ws-backend y arranca el rpc-backend
- Para en vm3 el ws-frontend y arranca el rpc-frontend
- Configura **RPCAPIBASEURL** en rpc-frontend para apuntar a vm2
- Verifica en navegador: **http://IP_VM3:8000/visaAppRPCFrontend/tarieta/**

```
./scripts/enable_threadgroup.sh P2-rdp  
./scripts/run_productivity_curve.sh P3-projects.jmx p2rdp_gunicorn
```

FASE 6: Ejercicio 6 - gunicorn vs runserver

Tiempo estimado: 15-20 min

6.1 Parar gunicorn en vm2

```
# En vm2, mata el proceso gunicorn (Ctrl+C si esta en primer plano)
# 0:
pkill gunicorn
```

6.2 Lanzar runserver

```
# En vm2, dentro de ~/P1-base/:
python manage.py runserver 0.0.0.0:8000
```

IMPORTANTE: Usa **0.0.0.0:8000**, NO **127.0.0.1:8000**. Sin 0.0.0.0 solo acepta conexiones locales y JMeter no podra conectarse.

6.3 Ejecutar curva de productividad con runserver

```
./scripts/run_productivity_curve.sh P3_P1-base.jmx p1base_runserver
```

6.4 Guardar resultados y volver a gunicorn

- Copia/apunta los datos de **results_p1base_runserver/throughput_summary.csv**
- Para runserver en vm2 (Ctrl+C)
- Vuelve a arrancar gunicorn para los siguientes ejercicios

```
gunicorn visaSite.wsgi:application --bind 0.0.0.0:8000
```

FASE 7: Ejercicio 7 - 2 workers + 2 CPUs

Tiempo estimado: 20-30 min

7.1 Apagar vm2

```
# En vm2:  
sudo shutdown -h now
```

7.2 Asignar 2 CPUs a vm2 en VirtualBox

- En VirtualBox, selecciona vm2 (apagada)
- Settings > System > Processor
- Cambia **Processor(s)** de 1 a 2
- Acepta y arranca vm2

7.3 Cambiar SESSION_ENGINE a 'db'

Esto es OBLIGATORIO con 2 workers. Con cache, cada worker tiene su propia cache y la sesion se pierde si el segundo request va a otro worker.

```
# En vm2, edita settings.py de P1-base:  
# Cambia:  
# SESSION_ENGINE = 'django.contrib.sessions.backends.cache'  
# Por:  
# SESSION_ENGINE = 'django.contrib.sessions.backends.db'  
  
# Puedes usar sed:  
sed -i 's/sessions.backends.cache/sessions.backends.db/'  
~/P1-base/visaSite/settings.py
```

7.4 Arrancar gunicorn con 2 workers

```
# En vm2:  
cd ~/P1-base/  
gunicorn visaSite.wsgi:application --bind 0.0.0.0:8000 --workers 2
```

7.5 Ejecutar curva de productividad

```
./scripts/run_productivity_curve.sh P3_P1-base.jmx p1base_2workers_2cpus
```

7.6 Guardar resultados

- Copia **results_p1base_2workers_2cpus/throughput_summary.csv**
- Compara con la curva de 1 worker - deberia ser significativamente mejor

FASE 8: Ejercicio 3 - Test de estres 1000 threads

Tiempo estimado: 15-20 min

8.1 Volver a 1 CPU y 1 worker (estado original)

- Apaga vm2, vuelve a poner 1 CPU en VirtualBox, arranca vm2
- Cambia SESSION_ENGINE de vuelta a cache
- Arranca unicorn con 1 worker (sin --workers)

```
gunicorn visaSite.wsgi:application --bind 0.0.0.0:8000
```

8.2 Instalar y arrancar nmon en las VMs

```
# En vm1 y vm2 (por SSH):  
sudo apt install nmon # si no esta instalado  
nmon
```

Con nmon abierto, pulsa: **c** (CPU), **m** (memoria), **d** (disco), **n** (red). Esto muestra los contadores en tiempo real.

8.3 Ejecutar el test de estres

```
./scripts/run_exercise3.sh
```

Esto ejecuta P3_P1-base.jmx con 1000 threads y Ramp-Up=1s.

8.4 Observar y capturar

IMPORTANTE: Mientras corre el test, OBSERVA nmon en vm1 y vm2. HAZ CAPTURAS DE PANTALLA de nmon mostrando el uso de CPU/memoria/red.

- Observa si la CPU llega al 100%
- Observa si la memoria se llena
- Observa la actividad de red
- Mira la salida de JMeter: cuantos errores hay?

8.5 Interpretar los resultados

Segun el enunciado, las tres hipotesis son:

- (a) JMeter no puede crear 1000 peticiones en 1 segundo
- (b) JMeter puede crearlas pero el throughput sera inferior
- (c) El servidor se desborda y pierde peticiones

La respuesta depende de lo que observes. Normalmente es (b) o (c).

FASE 9: Recoger datos y volver a casa

Tiempo estimado: 10 min

9.1 Checklist de datos recogidos

- Captura Aggregate Report del Ej1
- throughput_summary.csv de P1-base con gunicorn (Ej5)
- throughput_summary.csv de P1-ws con gunicorn (Ej5)
- throughput_summary.csv de P2-rcp con gunicorn (Ej5)
- throughput_summary.csv de P1-base con runserver (Ej6)
- throughput_summary.csv de P1-base con 2 workers + 2 CPUs (Ej7)
- Capturas de nmon durante test de estres (Ej3)
- Salida/log del test de estres (Ej3)

9.2 Copiar todo al USB

- Copia todas las carpetas **results_*** al USB
- Copia las capturas de pantalla
- Asegurate de tener los JMX finales (por si modificaste host/port)

9.3 Post-lab (en casa)

- Genera las graficas con **python3 scripts/generate_graphs.py**
- Rellena las secciones [PENDIENTE] de generate_memoria.py con los datos reales
- Regenera la memoria: **python3 generate_memoria.py**
- Prepara el ZIP para Moodle: memoria.pdf + JMX + codigo fuente

Referencia rapida de comandos

Accion	Comando
Arrancar gunicorn (1 worker)	<code>gunicorn visaSite.wsgi:application --bind 0.0.0.0:8000</code>
Arrancar gunicorn (2 workers)	<code>gunicorn visaSite.wsgi:application --bind 0.0.0.0:8000 --workers 2</code>
Arrancar runserver	<code>python manage.py runserver 0.0.0.0:8000</code>
Migraciones	<code>python manage.py migrate</code>
Poblar BD	<code>python manage.py populate</code>
Ver IP de la VM	<code>hostname -I</code>
Verificar PostgreSQL	<code>sudo systemctl status postgresql</code>
Matar gunicorn	<code>kill gunicorn</code>
JMeter CLI basico	<code>jmeter.sh -n -t file.jmx -Jusers=10 -Jsamples=10 -l r.jtl -e -o output/</code>
Habilitar Thread Group	<code>./scripts/enable_threadgroup.sh P1-ws</code>
Curva productividad	<code>./scripts/run_productivity_curve.sh <jmx> <etiqueta></code>
nmon	<code>nmon</code> (luego pulsa c, m, d, n)

Puertos por proyecto

Proyecto	Entrypoint URL	Puerto VM	Port Forward (si aplica)
P1-base	/visaApp/	8000	18000
P1-ws	/visaAppWSFrontend/	8000 (vm3)	18003
P2-rcp	/visaAppRPCFrontend/	8000 (vm3)	18005